Auth0 for AI Agents – Complete Setup Guide

Stack: Vercel AI SDK + Next.js + Token Vault

Part 1: Create Your Auth0 Account & Tenant

1.1 Sign up

1. Go to https://auth0.com/signup
2. Create a free account (the free tier includes Token Vault with 2 connected apps)
3. Choose a tenant name (e.g., `my-hackathon`) — this becomes your `AUTH0_DOMAIN`: `my-hackathon.us.auth0.com`

1.2 Create a Regular Web Application

1. In the Auth0 Dashboard → Applications → Applications → Create Application
2. Name: `Hackathon Agent` (or your project name)
3. Type: Regular Web Application
4. Go to Settings tab and note down:
   - `Domain` → this is your `AUTH0_DOMAIN`
   - `Client ID` → this is your `AUTH0_CLIENT_ID`
   - `Client Secret` → this is your `AUTH0_CLIENT_SECRET`
5. Under Allowed Callback URLs: `http://localhost:3000/auth/callback`
6. Under Allowed Logout URLs: `http://localhost:3000`
7. Under Advanced Settings → Grant Types: enable Token Exchange grant type
8. Save changes

1.3 Enable Refresh Token Rotation

1. In the same application Settings → scroll to Refresh Token Rotation
2. Enable Rotation
3. Under Application Properties → Application Type: ensure it says `Regular Web Application`

Part 2: Configure an External Connection (e.g., Google)

2.1 Create a Google OAuth connection

1. Dashboard → Authentication → Social → Create Connection
2. Select Google / Gmail
3. You'll need Google OAuth credentials:
   - Go to Google Cloud Console: https://console.cloud.google.com/
   - Create a project (or use existing)
   - APIs & Services → Credentials → Create OAuth 2.0 Client ID
   - Application type: Web application
   - Authorized redirect URI: `https://<YOUR_AUTH0_DOMAIN>/login/callback`
   - Copy the Client ID and Client Secret back into Auth0
4. Under Permissions (scopes), add the scopes your agent will need, e.g.:
   - `openid`, `profile`, `email`
   - `https://www.googleapis.com/auth/calendar.events` (for Google Calendar)
   - `https://www.googleapis.com/auth/gmail.readonly` (for Gmail)
5. Enable this connection for your application

2.2 Enable Token Vault for this connection

1. In the Google connection settings → Advanced tab
2. Toggle on Enable Token Vault
3. Save

> Repeat for other providers (GitHub, Slack, etc.) as needed.

Part 3: Set Up Your Next.js Project

3.1 Clone the sample app (recommended for hackathon speed)

```bash
```

```bash
# Option A: Clone the official sample
git clone https://github.com/auth0-samples/auth0-ai-samples.git
cd auth0-ai-samples/call-apis-on-users-behalf/others-api/vercel-ai-next-js

# Option B: Start fresh
npx create-next-app@latest my-agent-app --typescript --tailwind --app
cd my-agent-app
```

3.2 Install required packages

```bash
npm install @auth0/nextjs-auth0@4
npm install @auth0/ai-vercel@5 ai@6 @ai-sdk/openai@3 @ai-sdk/react@3 zod googleapis
```

3.3 Create your `.env.local` file

```env
# Auth0
APP_BASE_URL='http://localhost:3000'
AUTH0_SECRET='<run: openssl rand -hex 32>'
AUTH0_DOMAIN='your-tenant.us.auth0.com'
AUTH0_CLIENT_ID='your-client-id'
AUTH0_CLIENT_SECRET='your-client-secret'

# OpenAI (or any Vercel AI SDK supported provider)
OPENAI_API_KEY='your-openai-key'
```

Generate a secret:

```bash
openssl rand -hex 32
```

Part 4: Wire Up Authentication

4.1 Create the Auth0 client

```typescript
```

```
// src/lib/auth0.ts
import { Auth0Client } from '@auth0/nextjs-auth0/server';

export const auth0 = new Auth0Client({
  enableConnectAccountEndpoint: true,
});

export const getRefreshToken = async () => {
  const session = await auth0.getSession();
  return session?.tokenSet?.refreshToken;
};
```

4.2 Add authentication middleware

typescript

```
// src/middleware.ts
import { NextRequest, NextResponse } from "next/server";
import { auth0 } from "./lib/auth0";

export async function middleware(request: NextRequest) {
  const authRes = await auth0.middleware(request);

  if (request.nextUrl.pathname.startsWith("/auth")) {
    return authRes;
  }

  const { origin } = new URL(request.url);
  const session = await auth0.getSession(request);

  if (!session) {
    return NextResponse.redirect(`${origin}/auth/login`);
  }
  return authRes;
}

export const config = {
  matcher: [
    "/((?!_next/static|_next/image|images|favicon.[ico|png]|sitemap.xml|robots.txt|$
  ],
};
```

Part 5: Set Up Token Vault + Your First Tool

5.1 Configure the Auth0 AI SDK

typescript

```typescript
// src/lib/auth0-ai.ts
import { Auth0AI, getAccessTokenFromTokenVault } from "@auth0/ai-vercel";
import { getRefreshToken } from "./auth0";

export const getAccessToken = async () => getAccessTokenFromTokenVault();

const auth0AI = new Auth0AI();

// Create a wrapper for any Google connection tool
export const withGoogleConnection = auth0AI.withTokenVault({
  connection: "google-oauth2",
  scopes: ["openid", "https://www.googleapis.com/auth/calendar.events"],
  refreshToken: getRefreshToken,
});
```

5.2 Create a tool (e.g., Google Calendar)

typescript

```typescript
// src/lib/tools/google-calendar.ts
import { tool } from 'ai';
import { google } from 'googleapis';
import { z } from 'zod';
import { getAccessToken, withGoogleConnection } from '../auth0-ai';

export const getCalendarEventsTool = withGoogleConnection(
  tool({
    description: "Get calendar events for a given date",
    inputSchema: z.object({
      date: z.coerce.date(),
    }),
    execute: async ({ date }) => {
      const accessToken = await getAccessToken();

      const calendar = google.calendar('v3');
      const auth = new google.auth.OAuth2();
      auth.setCredentials({ access_token: accessToken });

      const response = await calendar.events.list({
        auth,
        calendarId: 'primary',
        timeMin: new Date(date).toISOString(),
        timeMax: new Date(new Date(date).getTime() + 86400000).toISOString(),
        singleEvents: true,
        orderBy: 'startTime',
        maxResults: 50,
      });

      return response.data.items?.map(event => ({
        summary: event.summary || 'No title',
        start: event.start?.dateTime || event.start?.date,
        end: event.end?.dateTime || event.end?.date,
      })) || [];
    },
  }),
);
```

5.3 Create the chat API route

```typescript
```

```
// src/app/api/chat/route.ts
import { openai } from '@ai-sdk/openai';
import { streamText } from 'ai';
import { setAIContext } from "@auth0/ai-vercel";
import { withInterruptions, errorSerializer } from "@auth0/ai-vercel/interrupts";
import { getCalendarEventsTool } from "@/lib/tools/google-calendar";

export async function POST(req: Request) {
  const { messages, id } = await req.json();
  setAIContext({ threadID: id });

  const tools = { getCalendarEventsTool };

  const result = streamText({
    model: openai('gpt-4o-mini'),
    system: 'You are a helpful calendar assistant.',
    messages,
    tools,
  });

  return result.toDataStreamResponse();
}
```

Part 6: Add Step-Up Authorization UI

When your agent tries to access Google Calendar for the first time, the user needs to consent. Auth0 handles this via "interrupts."

```bash
# Install the pre-built UI components
npx @auth0/ai-components add TokenVault
```

Then add the `TokenVaultInterruptHandler` component to your chat UI (see official quickstart for the full component code).

Part 7: Run & Test

```bash
```

```
npm run dev
```

1. Visit `http://localhost:3000`
2. You'll see the Auth0 Universal Login screen
3. Sign up / log in (use Google for best experience)
4. In the chat, type: "What's on my calendar today?"
5. The agent will trigger a consent popup → authorize Google Calendar access
6. Token Vault stores the token; subsequent calls work seamlessly

---

Quick Reference: Key Files

File	Purpose
`src/lib/auth0.ts`	Auth0 client + session helpers
`src/lib/auth0-ai.ts`	Token Vault SDK config
`src/middleware.ts`	Auth middleware (protects routes)
`src/lib/tools/*.ts`	Your AI agent tools (wrapped with Token Vault)
`src/app/api/chat/route.ts`	Chat API route (Vercel AI SDK)
`.env.local`	All secrets and config

Adding More Providers

To add GitHub, Slack, or any other provider:

1. Dashboard → Authentication → Social → Create Connection → select provider
2. Enable Token Vault in the connection's Advanced settings
3. In your code, create a new wrapper:

```typescript
```

```
export const withGitHubConnection = auth0AI.withTokenVault({
  connection: "github",
  scopes: ["public_repo", "read:user"],
  refreshToken: getRefreshToken,
});
```

4. Wrap your tool with it: `withGitHubConnection(tool({ ... }))`

Hackathon Tips

- Start with the sample app — don't build from scratch
- Use 2-3 connections max — Google + one more is enough to impress
- Implement step-up auth — judges will look for proper consent flows
- Show scopes clearly in your UI — scores well on "User Control" criteria
- Deploy to Vercel — free, fast, and the submission needs a live URL
- Write the bonus blog post — easy extra $250 opportunity